VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design with HDL (CO1026)

Assignment Report

Designing a simple RISC Processor

Instructor: Nguyễn Thành Lộc

Students:

Trần Vinh Quang	2550342
Lê Hiển Vinh	2453422
Nguyễn Đức Thiên Tân	2550345
Đặng Ngọc Lan Anh	2550258

HO CHI MINH CITY, JUNE 2026

Contents

1 Introduction **4**
1.1 Project Overview 4
1.2 Objectives 4
1.3 Tools and Environment 5
1.4 Main Features of the CPU 5
1.5 Report Organization 6

2 Theoretical Overview **6**
2.1 Fetch Phase 7
2.2 Execution Phase 7

3 System Design Approach **8**
3.1 Top-level System Architecture 8
3.2 Functional Blocks 9

4 Implementation **12**
4.1 Implementation Strategy 12
4.2 Program Counter (PC) 12
4.3 Address Multiplexer (Address MUX) 13
4.4 Arithmetic Logic Unit (ALU) 13
4.5 Controller Unit 14
4.6 Instruction Register (IR) 19
4.7 Accumulator Register (AC) 20
4.8 Memory 21
4.9 Top-level Integration 22

5 Testing and Evaluation **25**
5.1 Program Counter (PC) 25
5.2 Address Multiplexer (Address MUX) 29
5.3 Arithmetic Logic Unit (ALU) 33
5.4 Controller Unit 37
5.5 Instruction Register (IR) 43
5.6 Accumulator Register (AC) 46
5.7 Memory 50
5.8 Top-level Integration 53

6 Conclusion and Future Work **59**
6.1 Conclusion 59
6.2 Future Plan 59

7 Member Contribution **61**

1 Introduction

1.1 Project Overview

A Central Processing Unit (CPU) is the core component of a computer system, responsible for fetching instructions, decoding them, executing operations, and controlling data movement between internal registers and memory. In this project, a simple Reduced Instruction Set Computer (RISC) CPU is designed using *Verilog Hardware Description Language (HDL)*.

The proposed CPU follows a simplified RISC architecture with a compact instruction format. Each instruction consists of a 3-bit opcode and a 5-bit operand field, allowing the processor to support eight basic instructions and address a memory space of 32 locations. Although the architecture is simple, it demonstrates the fundamental principles of processor design, including instruction fetching, instruction decoding, arithmetic and logic execution, memory access, and program control.

The CPU is organized as a set of functional hardware modules, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and Controller. These modules are designed independently and then integrated into a complete processor system. The CPU operates based on clock and reset signals and continues executing instructions until a halt instruction is encountered.

1.2 Objectives

The main objective of this project is to design, implement, and verify a simple RISC CPU at the register-transfer level using Verilog HDL. Through this project, the design process of a basic processor can be studied from both theoretical and practical perspectives.

The specific objectives of the project are as follows:

- To understand the fundamental organization and operation of a RISC-based CPU.
- To design the main functional blocks of the CPU, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.
- To implement each functional block using Verilog HDL with a modular and hierarchical design approach.
- To integrate all modules into a complete CPU datapath and control path.
- To verify the correctness of each module through individual testbenches.
- To evaluate the complete CPU by observing simulation waveforms and checking the execution of supported instructions.

By completing these objectives, the project provides practical experience in digital system design, hardware modeling, finite state machine design, and simulation-based verification.

1.3 Tools and Environment

The main tool used in this project is Vivado Design Suite. Vivado provides an integrated development environment for designing, simulating, and verifying digital hardware systems described using Verilog HDL. In this project, Vivado is used to create Verilog source files, develop testbenches, run simulations, and analyze signal waveforms. The source code is formatted, linted, and maintained using `Verible` and the SystemVerilog Language Server (`SVLS`) to ensure strict hardware coding conventions.

Each CPU component is implemented as an independent Verilog module and verified using a corresponding testbench. A custom `Python` script is implemented with Icarus Verilog (`iverilog`) serves as the primary compiler for rapid, iterative command-line simulations and unit testing. After unit-level verification, the modules are connected together in a top-level design to perform system-level simulation. The waveform viewer in Vivado is used to observe the behavior of internal signals, such as the program counter value, memory address, instruction register output, accumulator value, ALU result, and controller signals.

Vivado is suitable for this project because it supports hierarchical hardware design, Verilog-based modeling, testbench simulation, and detailed waveform analysis. These features allow the design to be verified systematically before further hardware synthesis or implementation.

1.4 Main Features of the CPU

The designed CPU supports a small but representative set of instructions. Since the opcode field is 3 bits wide, the CPU can define up to 8 instructions. These instructions include program control, arithmetic operations, logic operations, memory access, and conditional execution.

The main features of the CPU are summarized as follows:

- A 3-bit opcode field supporting eight basic instructions.
- A 5-bit operand field supporting 32 memory address locations.
- A Program Counter used to store and update the current instruction address.
- An Address Multiplexer used to select between the instruction address and operand address.
- A Memory module used to store both instructions and data.
- An Instruction Register used to hold and decode the current instruction.

- An Accumulator Register used to store intermediate and final computation results.
- An ALU used to perform arithmetic and logic operations.
- A Controller implemented as a finite state machine to generate control signals for instruction execution.
- A halt mechanism that stops program execution when the halt instruction is detected.

The CPU performs its operation through a sequence of basic stages: instruction fetch, instruction decode, operand access, execution, and result storage. This execution flow reflects the essential behavior of a real processor while remaining simple enough for analysis, implementation, and verification.

1.5 Report Organization

This report is organized into six main sections.

Section 1 introduces the project, including the project overview, objectives, tools and environment, main CPU features, and report structure.

Section 2 presents the theoretical background of the project. It explains the basic concepts of RISC architecture, instruction format, CPU execution cycle, datapath, control path, and supported instructions.

Section 3 describes the design of the CPU. This section presents the overall system architecture, block diagram, and detailed functions of each hardware module.

Section 4 discusses the implementation of the CPU using Verilog HDL. It explains the module structure, implementation approach, controller finite state machine, memory organization, and top-level integration.

Section 5 presents the testing and evaluation process. It includes unit-level testbenches, system-level simulation, waveform analysis, and functional evaluation of the CPU.

Section 6 concludes the report by summarizing the achieved results, discussing difficulties encountered during the project, and proposing possible future improvements.

2 Theoretical Overview

Based on the project specification, the RISC CPU designed in this assignment features a 3-bit opcode and a 5-bit operand. This architecture supports 8 types of instructions (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP) and a 32-word memory address space. The processor operates synchronously with a clock signal and an active-high reset signal (the default reset state is `INST_ADDR`). Program execution stops upon encountering the `HLT` instruction.

By analyzing the controller state table and the functional requirements of each block, the CPU's operation flow can be divided into two primary phases spanning 8 clock cycles: the Fetch Phase and the Execution Phase.

Table 2.1: CPU Controller state and Control signals

Outputs	Phase								Notes
	INST_ADDR	INST_FETCH	INST_LOAD	IDLE	OP_ADDR	OP_FETCH	ALU_OP	STORE	
sel	1	1	1	1	0	0	0	0	ALU OP = 1 if opcode is ADD, AND, XOR or LDA
rd	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
ld_ir	0	0	1	1	0	0	0	0	
halt	0	0	0	0	HALT	0	0	0	
inc_pc	0	0	0	0	1	0	SKZ && zero	0	
ld_ac	0	0	0	0	0	0	0	ALUOP	
ld_pc	0	0	0	0	0	0	JMP	JMP	
wr	0	0	0	0	0	0	0	STO	
data_e	0	0	0	0	0	0	STO	STO	

2.1 Fetch Phase

- **State 1: INST_ADDR.** The Program Counter (PC) provides the address of the current instruction. The Address Mux selects the PC address (`sel = 1`) to access the memory.
- **State 2: INST_FETCH.** The memory read signal (`rd = 1`) is asserted. The instruction data is retrieved from Memory.
- **State 3: INST_LOAD.** The fetched instruction is loaded into the Instruction Register (IR) via the `ld_ir = 1` control signal.
- **State 4: IDLE.** An intermediate wait state for the system to stabilize.

2.2 Execution Phase

- **State 5: OP_ADDR.** The IR splits the instruction into a 3-bit Opcode and a 5-bit Operand Address. The Address Mux switches to select the operand address from the IR (`sel = 0`). Simultaneously, the PC increments (`inc_pc = 1`) to prepare for the next instruction cycle.

- **State 6: OP_FETCH.** If the instruction requires data from memory (e.g., ADD, AND, LDA), the CPU reads the data from the operand address in the Memory (`rd = 1`).
- **State 7: ALU_OP.** The Arithmetic Logic Unit (ALU) performs the required computation between the data fetched from Memory and the data currently held in the Accumulator (AC), based on the decoded Opcode. If the instruction is `SKZ` and the `zero` flag condition is met, the PC increments once more (`inc_pc = 1`) to skip the subsequent instruction.
- **State 8: STORE.** The result of the operation is either stored back into the Memory (for the `STO` instruction, asserting `wr = 1` and `data_e = 1`) or loaded into the Accumulator (for arithmetic and logic instructions, asserting `ld_ac = 1`).

3 System Design Approach

3.1 Top-level System Architecture

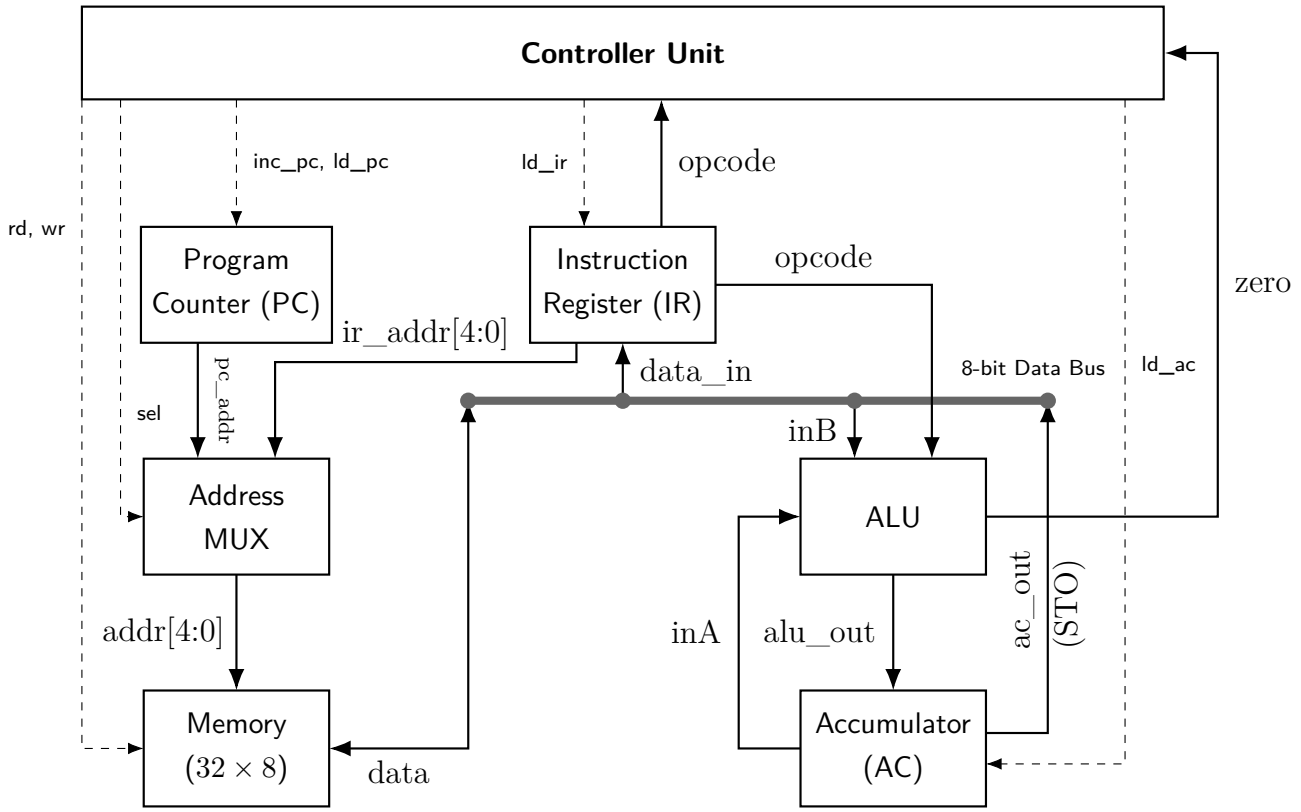

Figure 3.1: Top-Level Architecture of the RISC Processor

3.2 Functional Blocks

Program Counter (PC)

- **Function:**
  - The counter is a crucial component used to track and count program instructions.
  - It can also be utilized to count program states.
  - The counter operates synchronously on the rising edge of the `clk` signal.
  - The `reset` signal is active-high, clearing the counter back to 0.
  - The counter has a width of 5 bits.
  - It features a load function to force an arbitrary address into the counter. If not loading, the counter increments normally.
- **Inputs:** `clk`, `rst`, `ld_pc`, `inc_pc`, `data_in`.
- **Outputs:** `pc_out`.

Address Multiplexer (Address MUX)

- **Function:**
  - The Address MUX selects between the instruction address (during the instruction fetch phase) and the operand address (during the instruction execution phase).
  - The MUX has a default data width of 5 bits.
  - The width is parameterized to allow for easy modification if necessary.
- **Inputs:** `pc_addr`, `ir_addr`, `sel`.
- **Outputs:** `addr_out`.

Arithmetic Logic Unit (ALU)

- **Function:**
  - The ALU executes arithmetic and logical operations. The specific operation performed depends on the instruction's opcode.
  - The ALU executes 8 distinct operations on 8-bit operands (`inA` and `inB`).
  - It produces a 8-bit output and a 1-bit asynchronous `zero` flag.
  - The `zero` flag indicates whether the input `inA` is equal to 0.

- **Inputs:** `inA`, `inB`, `opcode`.
- **Outputs:** `alu_out`, `zero`.

Table 3.1: ALU Opcode Lookup Table

Opcode	Code	Operation Description	Output
`HLT`	000	Halts program execution.	`inA`
`SKZ`	001	Checks if the ALU result is equal to 0. If zero, it skips the next instruction; otherwise, execution continues normally.	`inA`
`ADD`	010	Adds the value in the Accumulator to the memory value at the instruction's address, returning the result to the Accumulator.	`inA + inB`
`AND`	011	Performs a bitwise AND on the Accumulator value and the memory value at the instruction's address.	`inA AND inB`
`XOR`	100	Performs a bitwise XOR on the Accumulator value and the memory value at the instruction's address.	`inA XOR inB`
`LDA`	101	Reads the value from the address in the instruction and loads it into the Accumulator.	`inB`
`STO`	110	Writes the Accumulator's data to the memory address specified in the instruction.	`inA`
`JMP`	111	Unconditional jump; jumps to the target address in the instruction and continues execution.	`inA`

Controller Unit

- **Function:** The central finite state machine that decodes instructions and dictates the operation of all other blocks via control signals.
- **Inputs:** `clk`, `rst`, `opcode`, `zero`.
- **Outputs:** `sel`, `rd`, `ld_ir`, `halt`, `inc_pc`, `ld_ac`, `ld_pc`, `wr`, `data_e`.

Table 3.2: Control Execution Scenarios by Opcode

Instruction Scenario	Active States	Expected Control Signals
HLT (000)	OP_ADDR	halt = 1
ADD, AND, XOR, LDA	OP_FETCH to STORE	rd = 1, ld_ac = 1 (at STORE phase)
STO (110)	ALU_OP, STORE	wr = 1, data_e = 1
JMP (111)	ALU_OP, STORE	ld_pc = 1

Instruction Register (IR)

- **Function:** As described in the reference documentation, it captures and holds the current instruction fetched from memory, splitting it into an opcode and an operand.
- **Inputs:** `clk`, `rst`, `ld_ir`, `data_in`.
- **Outputs:** `opcode`, `operand`.

Accumulator Register (AC)

- **Function:** As described in the reference documentation, it acts as the primary working register for ALU operations and data transfers.
- **Inputs:** `clk`, `rst`, `ld_ac`, `data_in`.
- **Outputs:** `ac_out`.

Memory

- **Function:**
  - The memory module stores both instructions and data.
  - It is designed to separate read and write functions while utilizing a single bidirectional data port. Reading and writing cannot occur simultaneously.
  - Configured for 5-bit addresses and 32-bit data.
  - Uses 1-bit control signals to enable reading (`rd`) and writing (`wr`).
  - Memory operations are synchronized to the rising edge of the `clk` signal.
- **Inputs:** `clk`, `rd`, `wr`, `addr`.
- **Inout:** `data`.

4 Implementation

4.1 Implementation Strategy

The implementation follows a bottom-to-up design flow consistent with industrial RTL practice. The process went through 4 phases: specification review, RTL coding for each individual module, functional simulation, and unit testing for debugging to ensure any failures happened only in the current development module and that the hardware matched the intended behavior.

Throughout the implementation, the following coding style guidelines were enforced for readable, synthesizable, and maintainable Verilog source code: maintaining consistent indentation, naming conventions (`lower_case` variable casing), and comprehensive comments throughout the code.

4.2 Program Counter (PC)

Implementation Logic & Steps:

- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the program counter.
- Implemented logic to load a new address into the program counter when `ld_pc` is active.
- Implemented logic to increment the program counter when `inc_pc` is active.

Source Code: `PC.v`:

```
module PC (
    input            clk,
    input            rst,
    input            ld_pc,
    input            inc_pc,
    input      [4:0] data_in,
    output reg [4:0] pc_out
);

  always @(posedge clk) begin
    if (rst) begin
      pc_out <= 5'd0;
    end else if (ld_pc) begin
      pc_out <= data_in;
```

```
    end else if (inc_pc) begin
      pc_out <= pc_out + 5'd1;
    end else begin
      pc_out <= pc_out;
    end
  end

endmodule
```

4.3 Address Multiplexer (Address MUX)

Implementation Logic & Steps:

- Implemented combinational logic to select between `pc_addr` and `ir_addr` based on the `sel` signal.
- Routed `pc_addr` to the output when `sel` is 1, and `ir_addr` when `sel` is 0.

Source Code: `ADD_MUX.v`:

```
module address_mux #(
    parameter WIDTH = 5
) (
    input  [WIDTH-1:0] pc_addr,
    input  [WIDTH-1:0] ir_addr,
    input              sel,
    output [WIDTH-1:0] addr_out
);

  assign addr_out = (sel) ? pc_addr : ir_addr;

endmodule
```

4.4 Arithmetic Logic Unit (ALU)

Implementation Logic & Steps:

- Implemented combinational logic to set the zero flag if `inA` is 0.

- Implemented combinational logic to perform arithmetic and logic operations based on the opcode.
- Defined operations for HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP instructions.
- Assigned the combinational result to `alu_out`.

Source Code: `ALU.v`:

```
module ALU (
    input       [2:0] opcode,
    input       [7:0] inA,
    input       [7:0] inB,
    output reg [7:0] alu_out,
    output           zero
);
  always @(*) begin
    case (opcode)
      3'b000:  alu_out = inA;
      3'b001:  alu_out = inA;
      3'b010:  alu_out = inA + inB;
      3'b011:  alu_out = inA & inB;
      3'b100:  alu_out = inA ^ inB;
      3'b101:  alu_out = inB;
      3'b110:  alu_out = inA;
      3'b111:  alu_out = inA;
      default: alu_out = 8'b00000000;
    endcase
  end
  assign zero = (inA == 8'b00000000);

endmodule
```

4.5 Controller Unit

Implementation Logic & Steps:

- Defined state parameters for the 8-state FSM.
- Defined opcode parameters for the 8 instructions.

- Declared a state register.
- Implemented a sequential logic block for FSM state transitions (triggered by positive clock edges).
- Implemented a synchronous active-high reset to set the initial state.
- Implemented combinational logic to determine output control signals based on the current state and opcode.
- Ensured all output signals were assigned a default value to prevent unwanted latches.

Source Code: `Controller.v`:

```
module controller (
    input            clk,
    input            rst,
    input      [2:0] opcode,
    input            zero,
    output reg       sel,
    output reg       rd,
    output reg       ld_ir,
    output reg       halt,
    output reg       inc_pc,
    output reg       ld_ac,
    output reg       ld_pc,
    output reg       wr,
    output reg       data_e

);
  localparam  INST_ADDR=3'd0,
              INST_FETCH=3'd1,
              INST_LOAD=3'd2,
              IDLE=3'd3,
              OP_ADDR=3'd4,
              OP_FETCH=3'd5,
              ALU_OP=3'd6,
              STORE=3'd7;

  // Opcodes
```

```
27    localparam HLT = 3'b000,
28               SKZ = 3'b001,
29               ADD = 3'b010,
30               AND = 3'b011,
31               XOR = 3'b100,
32               LDA = 3'b101,
33               STO = 3'b110,
34               JMP = 3'b111;
35
36    reg [2:0] state;
37    reg halted;
38
39    wire aluop;
40    assign aluop = (opcode == ADD) || (opcode == AND) || (opcode == XOR) ||
      (opcode == LDA);
41
42    // State register
43    always @(posedge clk) begin
44      if (rst) begin
45        state  <= INST_ADDR;
46        halted <= 1'b0;
47      end else if (halted) begin
48        state  <= state;
49        halted <= 1'b1;
50      end else begin
51        if ((state == OP_ADDR) && (opcode == HLT)) begin
52          state  <= OP_ADDR;
53          halted <= 1'b1;
54        end else begin
55          state <= (state == STORE) ? INST_ADDR : state + 3'd1;
56        end
57      end
58    end
59
60    // Output logic (combinational)
61    always @(*) begin
```

```
62      // Default values
63      sel    = 1'b0;
64      rd     = 1'b0;
65      ld_ir  = 1'b0;
66      halt   = 1'b0;
67      inc_pc = 1'b0;
68      ld_ac  = 1'b0;
69      ld_pc  = 1'b0;
70      wr     = 1'b0;
71      data_e = 1'b0;
72
73      if (rst) begin
74        sel = 1'b1;
75      end else if (halted) begin
76        halt = 1'b1;
77      end else begin
78        case (state)
79
80          INST_ADDR: begin
81            sel = 1'b1;
82          end
83
84          INST_FETCH: begin
85            sel = 1'b1;
86            rd  = 1'b1;
87          end
88
89          INST_LOAD: begin
90            sel   = 1'b1;
91            rd    = 1'b1;
92            ld_ir = 1'b1;
93          end
94
95          IDLE: begin
96            sel   = 1'b1;
97            rd    = 1'b1;
```

```
98            ld_ir = 1'b1;
99          end
100
101         OP_ADDR: begin
102           inc_pc = 1'b1;
103
104           if (opcode == HLT) begin
105             halt = 1'b1;
106           end
107         end
108
109         OP_FETCH: begin
110           rd = aluop;
111         end
112
113         ALU_OP: begin
114           rd = aluop;
115
116           if (opcode == SKZ && zero) begin
117             inc_pc = 1'b1;
118           end
119
120           if (opcode == JMP) begin
121             ld_pc = 1'b1;
122           end
123
124           if (opcode == STO) begin
125             data_e = 1'b1;
126           end
127         end
128
129         STORE: begin
130           rd    = aluop;
131           ld_ac = aluop;
132
133           if (opcode == JMP) begin
```

```
134             ld_pc = 1'b1;
135           end
136
137           if (opcode == STO) begin
138             wr     = 1'b1;
139             data_e = 1'b1;
140           end
141         end
142
143         default: begin
144           sel    = 1'b0;
145           rd     = 1'b0;
146           ld_ir  = 1'b0;
147           halt   = 1'b0;
148           inc_pc = 1'b0;
149           ld_ac  = 1'b0;
150           ld_pc  = 1'b0;
151           wr     = 1'b0;
152           data_e = 1'b0;
153         end
154
155       endcase
156     end
157   end
158
159 endmodule
```

4.6 Instruction Register (IR)

Implementation Logic & Steps:

- Declared an internal register to store the instruction data.
- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the register.
- Implemented logic to load `data_in` into the register when `ld_ir` is active.
- Implemented combinational logic to extract the 3-bit opcode from the stored instruction.

- Implemented combinational logic to extract the 5-bit operand from the stored instruction.

Source Code: `IR.v`:

```
module instruction_register (
    input        clk,
    input        rst,
    input        ld_ir,
    input  [7:0] data_in,
    output [2:0] opcode,
    output [4:0] operand
);

  reg [7:0] ir_reg;

  always @(posedge clk) begin
    if (rst) begin
      ir_reg <= 8'b00000000;
    end else if (ld_ir) begin
      ir_reg <= data_in[7:0];
    end
  end

  assign opcode  = ir_reg[7:5];

  assign operand = ir_reg[4:0];

endmodule
```

4.7 Accumulator Register (AC)

Implementation Logic & Steps:

- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the accumulator.
- Implemented logic to load new data into the accumulator when the load signal is active.

Source Code: `AC.v`:

```
module accumulator (
    input            clk,
    input            rst,
    input      [7:0] data_in,
    input            ld_ac,
    output reg [7:0] ac_out
);

  always @(posedge clk) begin
    if (rst) ac_out <= 8'b00000000;
    else if (ld_ac) ac_out <= data_in;
    else ac_out <= ac_out;
  end

endmodule
```

4.8 Memory

Implementation Logic & Steps:

- Declared a 2D array of registers to represent memory cells (32 cells of 8 bits).
- Declared a register to hold the output data.
- Implemented a sequential logic block triggered by the positive edge of the clock for writing data.
- Ensured writing only occurs when `wr` is active and `rd` is inactive.
- Implemented a sequential logic block triggered by the positive edge of the clock for reading data.
- Ensured reading only occurs when `rd` is active and `wr` is inactive.
- Implemented tri-state buffer logic for the bidirectional data bus.

Source Code: `Memory.v`:

```
module Memory (
    input       clk,
```

```
    input       rd,
    input       wr,
    input [4:0] addr,
    inout [7:0] data
);

  reg [7:0] mem_cells[31:0];
  reg [7:0] data_out;

  always @(posedge clk) begin
    if (wr && !rd) begin
      mem_cells[addr] <= data;
    end else if (rd && !wr) begin
      data_out <= mem_cells[addr];
    end
  end

  assign data = (rd && !wr) ? data_out : 8'hZZ;

endmodule
```

4.9 Top-level Integration

Implementation Logic & Steps:

- Declared internal wires for connecting all the CPU submodules.
- Instantiated the Program Counter (PC) module and connected its ports.
- Instantiated the Address Multiplexer (MUX) module and connected its ports.
- Instantiated the ALU module and connected its ports.
- Instantiated the Controller module and connected its ports.
- Instantiated the Instruction Register (IR) module and connected its ports.
- Instantiated the Accumulator (AC) module and connected its ports.
- Instantiated the Memory module and connected its ports.

- Implemented tri-state buffer logic for the data bus to handle writing from the Accumulator to Memory.

Source Code: `CPU.v`:

```
module CPU (
    input  clk,
    input  rst,
    output halt
);

  // Control signals
  wire sel;
  wire rd;
  wire ld_ir;
  wire inc_pc;
  wire ld_ac;
  wire ld_pc;
  wire wr;
  wire data_e;

  // Datapath signals
  wire [4:0] pc_out;
  wire [4:0] operand;
  wire [4:0] mem_addr;

  wire [2:0] opcode;

  wire [7:0] data_bus;
  wire [7:0] ac_out;
  wire [7:0] alu_out;

  wire zero;

  assign data_bus = (data_e && !rd) ? ac_out : 8'hZZ;

  PC u_pc (
      .clk    (clk),
```

```
        .rst    (rst),
        .ld_pc  (ld_pc),
        .inc_pc (inc_pc),
        .data_in(operand),
        .pc_out (pc_out)
    );

    address_mux #(
        .WIDTH(5)
    ) u_address_mux (
        .pc_addr (pc_out),
        .ir_addr (operand),
        .sel     (sel),
        .addr_out(mem_addr)
    );

    Memory u_memory (
        .clk (clk),
        .rd  (rd),
        .wr  (wr),
        .addr(mem_addr),
        .data(data_bus)
    );

    instruction_register u_ir (
        .clk    (clk),
        .rst    (rst),
        .ld_ir  (ld_ir),
        .data_in(data_bus),
        .opcode (opcode),
        .operand(operand)
    );

    accumulator u_ac (
        .clk    (clk),
        .rst    (rst),
```

```
70          .data_in(alu_out),
71          .ld_ac  (ld_ac),
72          .ac_out (ac_out)
73      );
74
75      ALU u_alu (
76          .opcode (opcode),
77          .inA    (ac_out),
78          .inB    (data_bus),
79          .alu_out(alu_out),
80          .zero   (zero)
81      );
82
83      controller u_controller (
84          .clk   (clk),
85          .rst   (rst),
86          .opcode(opcode),
87          .zero  (zero),
88          .sel   (sel),
89          .rd    (rd),
90          .ld_ir (ld_ir),
91          .halt  (halt),
92          .inc_pc(inc_pc),
93          .ld_ac (ld_ac),
94          .ld_pc (ld_pc),
95          .wr    (wr),
96          .data_e(data_e)
97      );
98
99  endmodule
```

5 Testing and Evaluation

5.1 Program Counter (PC)

test_001/PC_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// PC_tb.v - Testbench for Program Counter (PC)
// PC is 5-bit according to the assignment: 32 memory addresses.
// Covers:
//   TC1: Synchronous reset
//   TC2: Increment
//   TC3: Load
//   TC4: Priority rst > ld_pc > inc_pc
//   TC5: Hold
//   TC6: 5-bit wrap-around
//   TC7: Jump & increment
//   TC8: Reset from non-zero value
//   TC9: Data noise immunity
// ============================================================
module PC_tb;

  reg        clk;
  reg        rst;
  reg        ld_pc;
  reg        inc_pc;
  reg  [4:0] data_in;
  wire [4:0] pc_out;

  PC uut (
      .clk    (clk),
      .rst    (rst),
      .ld_pc  (ld_pc),
      .inc_pc (inc_pc),
      .data_in(data_in),
      .pc_out (pc_out)
  );

  initial clk = 0;
  always #5 clk = ~clk;
```

```
37
38    task tick;
39      input [63:0] cycle;
40      begin
41        @(posedge clk);
42        #1;
43        $display("cycle=%0d | rst=%b | ld_pc=%b | inc_pc=%b | data_in=%0d |
          pc_out=%0d", cycle, rst,
44                 ld_pc, inc_pc, data_in, pc_out);
45      end
46    endtask
47
48    initial begin
49      rst = 1;
50      ld_pc = 0;
51      inc_pc = 0;
52      data_in = 5'd0;
53
54      $display("--- TC1: Reset ---");
55      tick(1);  // pc_out = 0
56      rst = 0;
57
58      $display("--- TC2: Increment ---");
59      inc_pc = 1;
60      tick(2);  // pc_out = 1
61      tick(3);  // pc_out = 2
62      tick(4);  // pc_out = 3
63      inc_pc = 0;
64
65      $display("--- TC3: Load ---");
66      data_in = 5'd20;
67      ld_pc   = 1;
68      tick(5);  // pc_out = 20
69      ld_pc = 0;
70      tick(6);  // hold 20
71
```

```
72      $display("--- TC4: Priority rst > ld_pc > inc_pc ---");
73      data_in = 5'd25;
74      rst = 1;
75      ld_pc = 1;
76      inc_pc = 1;
77      tick(7);  // reset dominates -> 0
78      rst = 0;
79      tick(8);  // ld_pc dominates -> 25
80      ld_pc = 0;
81      tick(9);  // inc_pc -> 26
82      inc_pc = 0;
83
84      $display("--- TC5: Hold State ---");
85      tick(10);  // hold 26
86      tick(11);  // hold 26
87
88      $display("--- TC6: 5-bit Wrap-around ---");
89      data_in = 5'd31;
90      ld_pc   = 1;
91      tick(12);  // pc_out = 31
92      ld_pc  = 0;
93      inc_pc = 1;
94      tick(13);  // pc_out wraps to 0
95      inc_pc = 0;
96
97      $display("--- TC7: Jump & Increment ---");
98      data_in = 5'd10;
99      ld_pc   = 1;
100     tick(14);  // pc_out = 10
101     ld_pc  = 0;
102     inc_pc = 1;
103     tick(15);  // 11
104     tick(16);  // 12
105     inc_pc = 0;
106
107     $display("--- TC8: Reset from Non-Zero Value ---");
```

```
108     data_in = 5'd30;
109     ld_pc   = 1;
110     tick(17);  // pc_out = 30
111     ld_pc = 0;
112     rst   = 1;
113     tick(18);  // pc_out = 0
114     rst = 0;
115 
116     $display("--- TC9: Data Noise Immunity ---");
117     inc_pc = 1;
118     tick(19);  // pc_out = 1
119     data_in = 5'd31;
120     tick(20);  // pc_out = 2, not affected by data_in
121     inc_pc = 0;
122 
123     $display("--- DONE ---");
124     $finish;
125   end
126 
127 endmodule
```

5.2 Address Multiplexer (Address MUX)

test_002/AM_tb.v:

```
1  `timescale 1ns / 1ps
2  
3  // ============================================================
4  // AM_tb.v - Testbench for Address Mux
5  // Default WIDTH is 5 because CPU memory has 32 addresses.
6  // Covers:
7  //   TC1: Select pc_addr
8  //   TC2: Select ir_addr
9  //   TC3: Toggle sel
10 //   TC4: Parameter override
11 //   TC5: Stability
12 //   TC6: Edge case addresses
```

```
13 //   TC7: Async response
14 // ============================================================
15 module AM_tb;
16
17   reg  [4:0] pc_addr;
18   reg  [4:0] ir_addr;
19   reg        sel;
20   wire [4:0] addr_out;
21
22   address_mux uut (
23       .pc_addr (pc_addr),
24       .ir_addr (ir_addr),
25       .sel     (sel),
26       .addr_out(addr_out)
27   );
28
29   reg  [15:0] pc_addr_16;
30   reg  [15:0] ir_addr_16;
31   wire [15:0] addr_out_16;
32
33   address_mux #(
34       .WIDTH(16)
35   ) uut_16 (
36       .pc_addr (pc_addr_16),
37       .ir_addr (ir_addr_16),
38       .sel     (sel),
39       .addr_out(addr_out_16)
40   );
41
42   task display_state;
43     input [8*5:1] tc_name;
44     begin
45       #1;
46       $display("%s | sel=%b | pc_addr=%0d | ir_addr=%0d | addr_out=%0d",
         tc_name, sel, pc_addr,
47                ir_addr, addr_out);
```

```
48      end
49    endtask
50
51    initial begin
52      pc_addr = 5'd10;
53      ir_addr = 5'd20;
54      sel = 0;
55
56      pc_addr_16 = 16'd500;
57      ir_addr_16 = 16'd600;
58
59      $display("--- TC1: Select pc_addr ---");
60      sel = 1;
61      display_state("TC1  ");
62
63      $display("--- TC2: Select ir_addr ---");
64      sel = 0;
65      display_state("TC2  ");
66
67      $display("--- TC3: Toggle sel ---");
68      pc_addr = 5'd5;
69      ir_addr = 5'd25;
70      sel = 1;
71      display_state("TC3.1");
72      sel = 0;
73      display_state("TC3.2");
74      sel = 1;
75      display_state("TC3.3");
76
77      $display("--- TC4: Parameter Override (WIDTH=16) ---");
78      sel = 1;
79      #1;
80      $display("TC4.1 | sel=%b | pc_addr_16=%0d | ir_addr_16=%0d |
        addr_out_16=%0d", sel, pc_addr_16,
81              ir_addr_16, addr_out_16);
82      sel = 0;
```

```
83      #1;
84      $display("TC4.2 | sel=%b | pc_addr_16=%0d | ir_addr_16=%0d |
        addr_out_16=%0d", sel, pc_addr_16,
85              ir_addr_16, addr_out_16);
86
87      $display("--- TC5: Stability Check ---");
88      sel = 1;
89      pc_addr = 5'd7;
90      ir_addr = 5'd12;
91      display_state("TC5.1");
92      sel = 0;
93      pc_addr = 5'd15;
94      ir_addr = 5'd31;
95      display_state("TC5.2");
96
97      $display("--- TC6: Edge Case Addresses ---");
98      sel = 1;
99      pc_addr = 5'd0;
100     ir_addr = 5'd31;
101     display_state("TC6.1");
102     sel = 0;
103     display_state("TC6.2");
104
105     $display("--- TC7: Async Response ---");
106     sel = 1;
107     #1;
108     pc_addr = 5'd11;
109     display_state("TC7.1");
110     pc_addr = 5'd22;
111     display_state("TC7.2");
112     pc_addr = 5'd31;
113     display_state("TC7.3");
114
115     $display("--- DONE ---");
116     $finish;
117   end
```

```
118
119 endmodule
```

5.3 Arithmetic Logic Unit (ALU)

test_003/ALU_tb.v:

```
1  `timescale 1ns / 1ps
2
3  // ============================================================
4  // ALU_tb.v - Testbench for Arithmetic Logic Unit (ALU)
5  // Covers 7 scenarios:
6  //   TC1: Zero Status Flag
7  //   TC2: Arithmetic & Logic
8  //   TC3: Transfer Operations
9  //   TC4: Combinational Property
10 //   TC5: Boundary Cases
11 //   TC6: Complex Bitwise Logic
12 //   TC7: Zero Flag Independence
13 // ============================================================
14 module ALU_tb;
15
16   reg  [7:0] inA;
17   reg  [7:0] inB;
18   reg  [2:0] opcode;
19
20   wire [7:0] alu_out;
21   wire       zero;
22
23   // DUT
24   ALU uut (
25       .inA    (inA),
26       .inB    (inB),
27       .opcode (opcode),
28       .alu_out(alu_out),
29       .zero   (zero)
30   );
```

```
31
32    task display_state;
33      input [8*6:1] tc_name;
34      begin
35        #1;
36        $display("%s | opcode=%b | inA=%0d | inB=%0d | zero=%b | alu_out=%0d",
          tc_name, opcode, inA,
37                  inB, zero, alu_out);
38      end
39    endtask
40
41    initial begin
42
43      // Initialize
44      inA = 0;
45      inB = 0;
46      opcode = 3'b000;
47      #10;
48
49      // TC1: Zero Status Flag
50      $display("\n--- TC1: Zero Status Flag ---");
51
52      inA = 8'd0;
53      display_state("TC1.1");  // zero = 1
54
55      inA = 8'd50;
56      display_state("TC1.2");  // zero = 0
57
58      // TC2: Arithmetic & Logic Operations
59      $display("\n--- TC2: Arithmetic & Logic ---");
60
61      inA = 8'd100;
62      inB = 8'd25;
63
64      // ADD
65      opcode = 3'b010;
```

```
66      display_state("ADD");  // 125
67
68      // AND
69      opcode = 3'b011;
70      display_state("AND");  // 0
71
72      // XOR
73      opcode = 3'b100;
74      display_state("XOR");  // 125
75
76      // LDA
77      opcode = 3'b101;
78      display_state("LDA");  // 25
79
80
81      // TC3: Transfer Instructions
82      $display("\n--- TC3: HLT / SKZ / STO / JMP ---");
83
84      inA = 8'd200;
85      inB = 8'd100;
86
87      // HLT
88      opcode = 3'b000;
89      display_state("HLT");  // 200
90
91      // SKZ
92      opcode = 3'b001;
93      display_state("SKZ");  // 200
94
95      // STO
96      opcode = 3'b110;
97      display_state("STO");  // 200
98
99      // JMP
100     opcode = 3'b111;
101     display_state("JMP");  // 200
```

```
102
103
104     // TC4: Combinational Property
105     $display("\n--- TC4: Combinational Property ---");
106
107     opcode = 3'b010;  // ADD
108
109     inA = 8'd10;
110     inB = 8'd20;
111     #1;
112
113     $display("TC4.1 | alu_out=%0d (Expected 30)", alu_out);
114
115     // Change inputs immediately
116     inA = 8'd50;
117     inB = 8'd10;
118     #1;
119
120     $display("TC4.2 | alu_out=%0d (Expected 60)", alu_out);
121
122     // TC5: Boundary Cases
123     $display("\n--- TC5: Boundary Cases ---");
124
125     opcode = 3'b010;  // ADD
126
127     inA = 8'hFF;
128     inB = 8'h01;
129     display_state("OVF1");  // Expect 0
130
131     inA = 8'h7F;
132     inB = 8'h7F;
133     display_state("OVF2");  // Expect 254
134
135     // TC6: Complex Bitwise Logic
136     $display("\n--- TC6: Complex Bitwise Logic ---");
137
```

```
138     inA = 8'hAA;  // 10101010
139     inB = 8'h55;  // 01010101
140 
141     opcode = 3'b011;
142     display_state("AND_C");  // 0
143 
144     opcode = 3'b100;
145     display_state("XOR_C");  // 255
146 
147 
148     // TC7: Zero Flag Independence
149     $display("\n--- TC7: Zero Flag Independence ---");
150 
151     opcode = 3'b101;  // LDA
152 
153     inA = 8'd0;
154     inB = 8'd100;
155     #1;
156     $display("TC7.1 | zero=%b | alu_out=%0d", zero, alu_out);
157 
158     inA = 8'd1;
159     inB = 8'd100;
160     #1;
161     $display("TC7.2 | zero=%b | alu_out=%0d", zero, alu_out);
162 
163     $display("\n--- ALL TESTS FINISHED ---");
164     $finish;
165 
166   end
167 
168 endmodule
```

5.4 Controller Unit

test_004/Controller_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// Controller_tb.v - Testbench for Controller
// Covers:
//   TC1: Reset
//   TC2: Fetch phase
//   TC3: ADD execution
//   TC4: STO execution
//   TC5: JMP execution
//   TC6: HLT stable halt
//   TC7: SKZ with zero=0 and zero=1
// ============================================================
module Controller_tb;

  reg       clk;
  reg       rst;
  reg [2:0] opcode;
  reg       zero;

  wire sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e;

  controller uut (
      .clk   (clk),
      .rst   (rst),
      .opcode(opcode),
      .zero  (zero),
      .sel   (sel),
      .rd    (rd),
      .ld_ir (ld_ir),
      .halt  (halt),
      .inc_pc(inc_pc),
      .ld_ac (ld_ac),
      .ld_pc (ld_pc),
      .wr    (wr),
      .data_e(data_e)
```

```
37    );
38
39    initial clk = 0;
40    always #5 clk = ~clk;
41
42    task tick;
43      begin
44        @(posedge clk);
45        #1;
46      end
47    endtask
48
49    task reset_controller;
50      begin
51        rst = 1;
52        repeat (2) @(posedge clk);
53        #1;
54        rst = 0;
55      end
56    endtask
57
58    task display_state;
59      input [8*14:1] state_name;
60      begin
61        $display(
62            "%s | state=%3b opcode=%3b zero=%b | sel=%b rd=%b ld_ir=%b halt=%b
              inc_pc=%b ld_ac=%b ld_pc=%b wr=%b data_e=%b",
63            state_name, uut.state, opcode, zero, sel, rd, ld_ir, halt, inc_pc,
              ld_ac, ld_pc, wr,
64            data_e);
65      end
66    endtask
67
68    initial begin
69      opcode = 3'b000;
70      zero   = 1'b0;
```

```
71      rst    = 1'b1;
72
73      reset_controller();
74
75      $display("--- TC1: System Reset / INST_ADDR ---");
76      display_state("0:INST_ADDR   ");
77
78      $display("--- TC2: Fetch Phase ---");
79      opcode = 3'b010;  // ADD
80      display_state("0:INST_ADDR   ");
81      tick();
82      display_state("1:INST_FETCH  ");
83      tick();
84      display_state("2:INST_LOAD   ");
85      tick();
86      display_state("3:IDLE        ");
87
88      $display("--- TC3: Execution Phase (ADD) ---");
89      tick();
90      display_state("4:OP_ADDR     ");
91      tick();
92      display_state("5:OP_FETCH    ");
93      tick();
94      display_state("6:ALU_OP      ");
95      tick();
96      display_state("7:STORE       ");
97      tick();
98      display_state("0:INST_ADDR   ");
99
100     $display("--- TC4: Execution Phase (STO) ---");
101     reset_controller();
102     opcode = 3'b110;  // STO
103     tick();
104     tick();
105     tick();
106     tick();
```

```
107     display_state("4:OP_ADDR     ");
108     tick();
109     display_state("5:OP_FETCH    ");
110     tick();
111     display_state("6:ALU_OP      ");
112     tick();
113     display_state("7:STORE       ");
114 
115     $display("--- TC5: Execution Phase (JMP) ---");
116     reset_controller();
117     opcode = 3'b111;  // JMP
118     tick();
119     tick();
120     tick();
121     tick();
122     display_state("4:OP_ADDR     ");
123     tick();
124     display_state("5:OP_FETCH    ");
125     tick();
126     display_state("6:ALU_OP      ");
127     tick();
128     display_state("7:STORE       ");
129 
130     $display("--- TC6: Execution Phase (HLT, stable halt) ---");
131     reset_controller();
132     opcode = 3'b000;  // HLT
133     tick();
134     tick();
135     tick();
136     tick();
137     display_state("4:OP_ADDR/HLT ");
138     tick();
139     display_state("HOLD_HALT_1   ");
140     tick();
141     display_state("HOLD_HALT_2   ");
142 
```

```
143     $display("--- TC7.1: SKZ zero=0, no skip ---");
144     reset_controller();
145     opcode = 3'b001;  // SKZ
146     zero   = 1'b0;
147     tick();
148     tick();
149     tick();
150     tick();
151     display_state("4:OP_ADDR     ");
152     tick();
153     display_state("5:OP_FETCH    ");
154     tick();
155     display_state("6:ALU_OP      ");  // inc_pc should be 0
156     tick();
157     display_state("7:STORE       ");
158 
159     $display("--- TC7.2: SKZ zero=1, skip next instruction ---");
160     reset_controller();
161     opcode = 3'b001;  // SKZ
162     zero   = 1'b1;
163     tick();
164     tick();
165     tick();
166     tick();
167     display_state("4:OP_ADDR     ");
168     tick();
169     display_state("5:OP_FETCH    ");
170     tick();
171     display_state("6:ALU_OP      ");  // inc_pc should be 1
172     tick();
173     display_state("7:STORE       ");
174 
175     $display("--- DONE ---");
176     $finish;
177   end
178 
179 endmodule
```

5.5 Instruction Register (IR)

test_005/IR_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// IR_tb.v - Testbench for Instruction Register (IR)
// Covers 7 scenarios described in the project spec:
//   TC1: System Reset
//   TC2: Load Instruction
//   TC3: Hold Value
//   TC4: Extract Opcode and Operand
//   TC5: Reset Priority
//   TC6: Bus Noise Immunity
//   TC7: Instruction Decoding
// ============================================================

module IR_tb;
  // - Signals ---------------------
  reg clk;
  reg rst;
  reg ld_ir;
  reg [7:0] data_in;

  wire [2:0] opcode;
  wire [4:0] operand;

  // - DUT instantiation ---------------
  instruction_register uut (
      .clk    (clk),
      .rst    (rst),
      .ld_ir  (ld_ir),
      .data_in(data_in),
      .opcode (opcode),
      .operand(operand)
  );
```

```
34
35    // - Clock: 10 ns period ---------------
36    initial begin
37      clk = 0;
38      forever #5 clk = ~clk;
39    end
40
41    // - Task: tick one clock and display state ------
42    task display_state;
43      input [8*5:1] tc_name;
44      begin
45        @(posedge clk);
46        #1;  // Wait after posedge for data to update
47        $display("%s | rst=%b | ld_ir=%b | data_in=0x%02X | opcode=%b |
          operand=%b", tc_name, rst,
48                 ld_ir, data_in, opcode, operand);
49      end
50    endtask
51
52    initial begin
53      // Init
54      rst = 1;
55      ld_ir = 0;
56      data_in = 8'd0;
57      #1;
58      // - TC1: System Reset --------------
59      $display("--- TC1: System Reset ---");
60      display_state("TC1.1");  // Expect opcode=0, operand=0
61
62      rst = 0;  // De-assert reset
63
64      // - TC2: Load Instruction ------------
65      $display("--- TC2: Load Instruction ---");
66      ld_ir   = 1;
67      // Instruction 1: Opcode = 101 (LDA), Operand = 10101
68      // Combined: [7:5] = 101, [4:0] = 10101 -> 1011_0101 = 0xB5
```

```
69      data_in = 8'hB5;
70      display_state("TC2.1");  // Expect: opcode=101, operand=10101
71
72      // Instruction 2: Opcode = 010 (ADD), Operand = 00011
73      // Combined: [7:5] = 010, [4:0] = 00011 -> 0100_0011 = 0x43
74      data_in = 8'h43;
75      display_state("TC2.2");  // Expect: opcode=010, operand=00011
76
77      // - TC3: Hold Value ---------------
78      $display("--- TC3: Hold Value ---");
79      ld_ir   = 0;  // De-assert load instruction signal
80      data_in = 8'hFF;  // Change data on bus
81      display_state("TC3.1");  // Expect: opcode=010, operand=00011 (holds
        value)
82      display_state("TC3.2");  // Expect: opcode=010, operand=00011
83
84      // - TC4: Extract Opcode and Operand -------
85      $display("--- TC4: Extract Opcode and Operand ---");
86      // Instruction 3: Opcode = 111 (JMP), Operand = 11111
87      // Combined: [7:5] = 111, [4:0] = 11111 -> 1111_1111 = 0xFF
88      ld_ir   = 1;
89      data_in = 8'hFF;
90      display_state("TC4.1");  // Expect: opcode=111, operand=11111
91
92      // Instruction 4: Opcode = 000 (HLT), Operand = 00000
93      data_in = 8'h00;
94      display_state("TC4.2");  // Expect: opcode=000, operand=00000
95
96      // - TC5: Reset Priority -------------
97      $display("--- TC5: Reset Priority ---");
98      ld_ir   = 1;
99      data_in = 8'hFF;  // Attempt to load all 1s
100     rst     = 1;  // But assert reset simultaneously
101     display_state("TC5.1");  // Expect: opcode=000, operand=00000
102     rst = 0;
```

```
103     display_state("TC5.2");  // After reset is de-asserted, FF data is
        loaded (opcode=111, op=11111)
104
105     // - TC6: Bus Noise Immunity -----------
106     // Change data_in continuously when ld_ir = 0
107     $display("--- TC6: Bus Noise Immunity ---");
108     ld_ir   = 0;
109     data_in = 8'hAA;
110     #2;
111     data_in = 8'h55;
112     #2;
113     data_in = 8'h78;
114     display_state("TC6.1");  // Expect: holds value from TC5.2 (111, 11111)
115
116     // - TC7: Instruction Decoding ----------
117     $display("--- TC7: Instruction Decoding ---");
118     ld_ir   = 1;
119     // Load ADD (010) Operand (01010)
120     data_in = 8'h4A;  // 4A = 010_01010
121     display_state("TC7.1");  // Expect: opcode=010, operand=01010
122
123     // Load LDA (101) Operand (11111)
124     data_in = 8'hBF;  // BF = 101_11111
125     display_state("TC7.2");  // Expect: opcode=101, operand=11111
126
127     $display("--- DONE ---");
128     $finish;
129   end
130 endmodule
```

5.6 Accumulator Register (AC)

test_006/AC_tb.v:

```
1 `timescale 1ns / 1ps
2
3 // ============================================================
```

```
// AC_tb.v - Testbench for Accumulator (AC)
// Covers 8 scenarios:
//   TC1: System Reset
//   TC2: Load Data
//   TC3: Data Stability
//   TC4: Feedback to ALU
//   TC5: Interaction with Data Bus
//   TC6: Reset Priority
//   TC7: Boundary Values
//   TC8: Rapid Toggling
// ============================================================
module AC_tb;

  // - Signals ---------------------
  reg        clk;
  reg        rst;
  reg        ld_ac;
  reg  [7:0] data_in;
  wire [7:0] ac_out;

  // - DUT instantiation ----------------
  accumulator uut (
      .clk    (clk),
      .rst    (rst),
      .ld_ac  (ld_ac),
      .data_in(data_in),
      .ac_out (ac_out)
  );

  // - Clock: 10 ns period ---------------
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  // - Task: tick one clock and display state ------
```

```
40    task display_state;
41      input [8*5:1] tc_name;
42      begin
43        @(posedge clk);
44        #1;  // Wait after posedge for data to stabilize
45        $display("%s | rst=%b | ld_ac=%b | data_in=%0d | ac_out=%0d", tc_name,
          rst, ld_ac, data_in,
46                 ac_out);
47      end
48    endtask
49
50    initial begin
51      // Init
52      rst = 1;
53      ld_ac = 0;
54      data_in = 8'd0;
55      #1;
56
57      // - TC1: System Reset --------------
58      $display("--- TC1: System Reset ---");
59      display_state("TC1  ");  // Expect: ac_out = 0
60      rst = 0;
61
62      // - TC2: Load Data (ALU Result) ---------
63      $display("--- TC2: Load Data (ALU Result) ---");
64      ld_ac   = 1;
65      data_in = 8'd150;  // Assume result from ALU is 150
66      display_state("TC2.1");  // Expect: ac_out = 150
67
68      data_in = 8'd200;
69      display_state("TC2.2");  // Expect: ac_out = 200
70
71      // - TC3: Data Stability (Idle/Fetch) -------
72      $display("--- TC3: Data Stability (Idle/Fetch) ---");
73      ld_ac   = 0;
74      data_in = 8'd255;  // Change input data
```

```
75      display_state("TC3.1");  // Expect: ac_out holds 200
76      display_state("TC3.2");  // Expect: ac_out holds 200
77
78      // - TC4: Feedback to ALU -------------
79      $display("--- TC4: Feedback to ALU ---");
80      // This case verifies that ac_out (inA of ALU) is always ready
81      display_state("TC4  ");  // Expect: ac_out = 200
82
83      // - TC5: Interaction with Data Bus --------
84      $display("--- TC5: Interaction with Data Bus ---");
85      // Simulate loading a new value in preparation to write to Memory
86      ld_ac   = 1;
87      data_in = 8'd100;
88      display_state("TC5.1");  // Expect: ac_out = 100
89      ld_ac = 0;
90      display_state("TC5.2");  // Value holds stable during write cycle
91
92      // - TC6: Reset Priority -------------
93      $display("--- TC6: Reset Priority Check ---");
94      ld_ac   = 1;
95      data_in = 8'd123;
96      rst     = 1;  // Assert Reset while load is active
97      display_state("TC6.1");  // Expect: ac_out = 0 (Reset wins)
98      rst = 0;
99      display_state("TC6.2");  // Expect: ac_out = 123 (Normal load resumes)
100
101     // - TC7: Boundary Values -------------
102     $display("--- TC7: Boundary Values ---");
103     ld_ac   = 1;
104     data_in = 8'hFF;  // All 1s
105     display_state("TC7.1");
106     data_in = 8'h00;  // All 0s
107     display_state("TC7.2");
108
109     // - TC8: Rapid Toggling -------------
110     $display("--- TC8: Rapid Toggling ---");
```

```
    ld_ac   = 1;
    data_in = 8'hAA;
    display_state("TC8.1");
    data_in = 8'h55;
    display_state("TC8.2");
    ld_ac   = 0;
    data_in = 8'h12;
    display_state("TC8.3");  // Expect: Holds 8'h55

    $display("--- DONE ---");
    $finish;
  end
endmodule
```

5.7 Memory

test_007/Memory_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// Memory_tb.v - Testbench for Memory
// Covers 5 scenarios described in the project spec:
//   TC1: Write Data
//   TC2: Read Data
//   TC3: High Impedance Check
//   TC4: Conflict Prevention
//   TC5: Data Persistence
// ============================================================
module Memory_tb;

  // - Signals ---------------------
  reg        clk;
  reg        rd;
  reg        wr;
  reg  [4:0] addr;  // 5-bit address for 32 Memory cells
  wire [7:0] data;  // Bidirectional bus
```

```
20
21    // Bus control signals from Testbench
22    reg  [7:0] data_reg;
23
24    // Inout control logic: Write only when wr=1 and rd=0
25    assign data = (wr && !rd) ? data_reg : 8'hZZ;
26
27    // - DUT instantiation ----------------
28    Memory uut (
29        .clk (clk),
30        .rd  (rd),
31        .wr  (wr),
32        .addr(addr),
33        .data(data)
34    );
35
36    // - Clock: 10 ns period ---------------
37    initial begin
38      clk = 0;
39      forever #5 clk = ~clk;
40    end
41
42    // - Task: tick one clock and display state ------
43    task display_state;
44      input [8*5:1] tc_name;
45      begin
46        @(posedge clk);
47        #1;  // Wait after posedge for data to stabilize
48        $display("%s | rd=%b | wr=%b | addr=%0d | data=0x%02X", tc_name, rd,
          wr, addr, data);
49      end
50    endtask
51
52    initial begin
53      // Init
54      rd       = 0;
```

```
55      wr       = 0;
56      addr     = 0;
57      data_reg = 0;
58      #10;
59
60      // - TC1: Write Data ---------------
61      $display("--- TC1: Write Data ---");
62      wr       = 1;
63      rd       = 0;
64      addr     = 5'd10;
65      data_reg = 8'hA5;
66      display_state("TC1.1");  // Write to Memory cell 10
67
68      addr     = 5'd20;
69      data_reg = 8'h12;
70      display_state("TC1.2");  // Write to Memory cell 20
71      wr = 0;
72
73      // - TC2: Read Data ----------------
74      $display("--- TC2: Read Data ---");
75      wr   = 0;
76      rd   = 1;
77      addr = 5'd10;
78      display_state("TC2.1");  // Expect: A5
79      addr = 5'd20;
80      display_state("TC2.2");  // Expect: 12
81
82      // - TC3: High Impedance Check ----------
83      $display("--- TC3: High Impedance Check ---");
84      wr   = 0;
85      rd   = 0;  // No read, no write
86      addr = 5'd10;
87      display_state("TC3  ");  // Expect: data = ZZ
88
89      // - TC4: Conflict Prevention -----------
90      $display("--- TC4: Conflict Prevention ---");
```

```
    // Per spec: Simultaneous read and write not allowed
    wr       = 1;
    rd       = 1;
    addr     = 5'd10;
    data_reg = 8'hFF;
    display_state("TC4  ");  // Expect: System must handle safely (usually
    high-Z or priority)

    // - TC5: Data Persistence ------------
    $display("--- TC5: Data Persistence ---");
    wr   = 0;
    rd   = 1;
    addr = 5'd10;
    display_state("TC5  ");  // Confirm data in cell 10 is still A5 after
    conflict

    $display("--- DONE ---");
    $finish;
  end
endmodule
```

5.8 Top-level Integration

test_008/CPU_tb.v:

```
`timescale 1ns / 1ps

// ============================================================
// CPU_tb.v - Top-Level Testbench for Integration (CPU)
//    - LDA 20
//    - ADD 21
//    - STO 22
//    - HLT
// ============================================================
module CPU_tb;

  // - Signals --------------------
```

```
13    reg  clk;
14    reg  rst;
15    wire halt;
16
17    // - DUT instantiation ----------------
18    CPU uut (
19        .clk (clk),
20        .rst (rst),
21        .halt(halt)
22    );
23
24    // - Clock: 10 ns period ---------------
25    initial clk = 0;
26    always #5 clk = ~clk;
27
28    // - Task: display CPU state -------------
29    task display_cpu_state;
30      begin
31        $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%b",
          $time, uut.u_pc.pc_out,
32                 uut.u_controller.state, uut.opcode, uut.u_ac.ac_out, halt);
33      end
34    endtask
35
36    // - Load Program & Control Simulation --------
37    initial begin
38      // 1. Load specific test program
39      // LDA 20, ADD 21, STO 22, HLT
40      uut.u_memory.mem_cells[0] = 8'hB4;  // LDA 20 (Opcode 101, Operand
        10100)
41      uut.u_memory.mem_cells[1] = 8'h55;  // ADD 21 (Opcode 010, Operand
        10101)
42      uut.u_memory.mem_cells[2] = 8'hD6;  // STO 22 (Opcode 110, Operand
        10110)
43      uut.u_memory.mem_cells[3] = 8'h00;  // HLT    (Opcode 000, Operand
        00000)
```

```
44
45      // 2. Load input data
46      uut.u_memory.mem_cells[20] = 8'd5;
47      uut.u_memory.mem_cells[21] = 8'd3;
48      uut.u_memory.mem_cells[22] = 8'd0;  // Location to store result
49
50      // 3. Reset system
51      rst = 1;
52      repeat (2) @(posedge clk);
53      #1;
54      rst = 0;
55
56      $display("--- CPU INTEGRATION TEST STARTED ---");
57    end
58
59    // Monitor state at each positive clock edge
60    always @(posedge clk) begin
61      if (!rst) display_cpu_state();
62    end
63
64    // Final result check after HLT
65    initial begin
66      wait (halt === 1'b1);
67      repeat (2) @(posedge clk);
68
69      $display("--- FINAL RESULT CHECK ---");
70      $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
71
72      if (uut.u_memory.mem_cells[22] == 8) $display("--- TEST STATUS: PASS
        ---");
73      else $display("--- TEST STATUS: FAIL ---");
74
75      $finish;
76    end
77
78    // Safety Timeout
```

```
79    initial #10000 $finish;
80
81  endmodule
```

test_009/CPU2_tb.v:

```
1   `timescale 1ns / 1ps
2
3   // ============================================================
4   // CPU2_tb.v - Top-Level Testbench for Integration (CPU)
5   // Covers execution of a comprehensive test program evaluating:
6   //    - LDA, ADD, STO sequence
7   //    - Conditional SKZ branching
8   //    - Unconditional JMP branching
9   //    - Bitwise XOR and AND operations
10  //    - HLT assertion
11  // ============================================================
12  module CPU2_tb;
13
14    // - Signals ---------------------
15    reg  clk;
16    reg  rst;
17    wire halt;
18
19    // - DUT instantiation ----------------
20    CPU uut (
21        .clk (clk),
22        .rst (rst),
23        .halt(halt)
24    );
25
26    // - Clock: 10 ns period ---------------
27    initial clk = 0;
28    always #5 clk = ~clk;
29
30    // - Task: display CPU state -------------
```

```
31    // Prints out whenever PC changes or an instruction cycle finishes (8
      phases)
32    task display_cpu_state;
33      begin
34        $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%b",
          $time, uut.u_pc.pc_out,
35                  uut.u_controller.state, uut.opcode, uut.u_ac.ac_out, halt);
36      end
37    endtask
38
39    // - Load Program & Control Simulation --------
40    initial begin
41      // 1. Load comprehensive test program into Memory
42      // Program executes: (5 + 3), XOR 5, AND 3, tests SKZ/JMP
43      uut.u_memory.mem_cells[0] = 8'hB4;  // LDA 20 -> AC = 5
44      uut.u_memory.mem_cells[1] = 8'h55;  // ADD 21 -> AC = 8
45      uut.u_memory.mem_cells[2] = 8'hD6;  // STO 22 -> Mem[22] = 8
46      uut.u_memory.mem_cells[3] = 8'h20;  // SKZ    -> AC=8 (!=0) -> No jump
47      uut.u_memory.mem_cells[4] = 8'hE6;  // JMP 6  -> PC jumps to 6
48      uut.u_memory.mem_cells[5] = 8'h55;  // ADD 21 -> (Skipped due to JMP)
49      uut.u_memory.mem_cells[6] = 8'h94;  // XOR 20 -> 8 XOR 5 = 13
50      uut.u_memory.mem_cells[7] = 8'h75;  // AND 21 -> 13 AND 3 = 1
51      uut.u_memory.mem_cells[8] = 8'hD7;  // STO 23 -> Mem[23] = 1
52      uut.u_memory.mem_cells[9] = 8'hB8;  // LDA 24 -> AC = 0 (Data at 24 is
        0)
53      uut.u_memory.mem_cells[10] = 8'h20;  // SKZ    -> AC=0 -> Skip next
        instruction
54      uut.u_memory.mem_cells[11] = 8'hD9;  // STO 25 -> (Skipped due to SKZ)
55      uut.u_memory.mem_cells[12] = 8'h00;  // HLT    -> Halt
56
57      // 2. Load operand data
58      uut.u_memory.mem_cells[20] = 8'd5;
59      uut.u_memory.mem_cells[21] = 8'd3;
60      uut.u_memory.mem_cells[22] = 8'd0;
61      uut.u_memory.mem_cells[23] = 8'd0;
62      uut.u_memory.mem_cells[24] = 8'd0;
```

```
63
64      // 3. Reset system
65      rst = 1;
66      repeat (2) @(posedge clk);
67      #1;
68      rst = 0;
69
70      $display("--- CPU INTEGRATION TEST STARTED ---");
71    end
72
73    // Monitor state at each positive clock edge
74    always @(posedge clk) begin
75      if (!rst) display_cpu_state();
76    end
77
78    // Final result check after HLT
79    initial begin
80      wait (halt === 1'b1);
81      repeat (2) @(posedge clk);
82
83      $display("--- FINAL RESULT CHECK ---");
84      $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
85      $display("Mem[23] (Expected 1): %0d", uut.u_memory.mem_cells[23]);
86
87      if (uut.u_memory.mem_cells[22] == 8 && uut.u_memory.mem_cells[23] == 1)
88        $display("--- TEST STATUS: PASS ---");
89      else $display("--- TEST STATUS: FAIL ---");
90
91      $finish;
92    end
93
94    // Safety Timeout
95    initial #10000 $finish;
96
97  endmodule
```

6 Conclusion and Future Work

6.1 Conclusion

In this assignment, our group designed and implemented a simple RISC processor using Verilog HDL. The processor follows an accumulator-based architecture and uses an 8-bit instruction format, consisting of a 3-bit opcode and a 5-bit operand field. Based on this format, the CPU supports eight instructions: HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP. The design also includes a 5-bit program counter and a 32-location memory space, which are consistent with the project specification.

The processor was developed using a modular design approach. Core components, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller, were implemented as independent Verilog modules and then integrated into the top-level CPU module. This organization made the datapath and control path clearer, while also making simulation, debugging, and system integration more manageable.

Key part of the design was the Controller, which was implemented as a finite state machine with eight states: `INST_ADDR`, `INST_FETCH`, `INST_LOAD`, `IDLE`, `OP_ADDR`, `OP_FETCH`, `ALU_OP`, and `STORE`. These states coordinate the main phases of instruction execution, including instruction fetch, decoding, operand access, ALU operation, accumulator update, memory write-back, program counter update, and halt control. Through simulation, the processor was verified to execute the supported instructions correctly at both module and system levels.

The main strength of the design is its clear separation between datapath components and control logic. Since each module was tested independently before being connected in the complete CPU, functional errors were easier to identify and correct. In addition, the use of both module-level and CPU-level testbenches helped confirm the correctness of individual blocks as well as the overall instruction execution flow.

However, the current processor still has several limitations. The instruction set is relatively small and does not support more complex arithmetic or comparison operations. The 5-bit address bus also limits the memory space to only 32 locations, which restricts the size of executable programs. Moreover, the controller follows a fixed multi-state execution sequence, meaning that some simple instructions may require more clock cycles than necessary. The project also focuses mainly on functional simulation, while synthesis results, timing delay, resource utilization, and power consumption have not yet been analyzed in detail.

6.2 Future Plan

Based on these limitations, several improvements can be considered for future development. First, the instruction set can be extended to improve the flexibility of the processor. Additional

instructions such as SUB, MUL, DIV, shift operations, and comparison instructions would allow the CPU to support more diverse computational tasks. For example, comparison and shift instructions would make the processor more capable of handling conditional operations and bit-level data manipulation.

Second, the memory system can be expanded by increasing the address width. Since the current 5-bit address bus only supports 32 memory locations, a wider address bus would allow the processor to access a larger program and data memory space. This improvement would make it possible to execute longer instruction sequences and store more intermediate data during program execution.

Third, the Controller can be optimized to reduce unnecessary execution cycles. In the current design, instructions follow a fixed sequence of states, even when some operations do not require all stages. Future versions could use instruction-dependent state transitions so that simpler instructions, such as HLT or JMP, complete in fewer cycles. More advanced control techniques, including pipelining and hazard handling, could also be explored to improve instruction throughput.

Fourth, verification should be extended with more comprehensive test programs. Although module-level and CPU-level simulations were performed, additional cases should be tested, such as repeated jump operations, consecutive store instructions, SKZ behavior when the accumulator is zero and non-zero, and boundary memory access at the lowest and highest addresses. These cases would help evaluate the robustness of the processor under less straightforward execution scenarios.

Finally, synthesis and implementation analysis should be performed to evaluate the hardware cost of the design. Resource utilization, critical path delay, maximum operating frequency, and power consumption should be analyzed after synthesis. This would provide a more complete understanding of the processor not only as a functional Verilog model, but also as a hardware design that could be implemented on an FPGA or similar digital platform.

Overall, this assignment helped us understand the fundamental principles of processor design, especially the interaction between datapath modules and control logic. It also provided practical experience in using Verilog HDL, designing a finite state machine controller, integrating multiple hardware modules, and verifying a digital system through simulation.

7 Member Contribution

Table 7.1: Member Contribution

No.	Member	Student ID	Contribution
1	Trần Vinh Quang	2550342	Implemented the Controller module, including the finite state machine and control signal generation. Contributed to the project report.
2	Lê Hiển Vinh	2453422	Implemented the Program Counter and top-level CPU modules. Developed the testbenches and expected output files, and performed simulation verification for the system.
3	Nguyễn Đức Thiên Tân	2550345	Implemented the Memory and Instruction Register modules, including memory access logic and instruction decoding. Contributed to the project report.
4	Đặng Ngọc Lan Anh	2550258	Implemented the ALU and Accumulator modules, including arithmetic, logic, zero flag, and register operations. Contributed to the project report.